Indian Institute of Technology Tirupati
AI Accelerator Design | Assignment 3

Instructor: Prof. Jaynarayan T. Tudu **Name:** Gurudeep | **Roll No:** CS25M111
Start Date: 2nd Oct 2026 | **Deadline:** 6th Oct 2026

1 Flattening and Reconstruction Algorithms

Given a four-dimensional input image or feature-map tensor in **BCHW** (or **NCHW**) format:

$$I \in \mathbb{R}^{B\times C\times H\times W},$$

where B is the batch size, C is the number of channels, H is the height, and W is the width, the two required layout transformations are:

$$B \times C \times H \times W \;\longrightarrow\; B \times (CHW) \;\longrightarrow\; B \times C \times H \times W.$$

1.1 Address Mapping Equations

In row-major BCHW layout, for each batch index $b \in \{0, \ldots, B-1\}$, the width W has unit stride (1), the height H has stride W, and the channel C has stride $H \cdot W$:

- **Forward Mapping (BCHW $\to B \times (CHW)$):** Each 4D index (b, c, h, w) maps to the 2D index (b, k) where $0 \le k < C \cdot H \cdot W$ is computed as:

$$k = c \cdot (H \cdot W) + h \cdot W + w. \tag{1}$$

- **Inverse Reconstruction ($B \times (CHW) \to$ BCHW):** Given the flattened index (b, k), the original indices (c, h, w) are recovered using integer division and modulo operations:

$$c = \left\lfloor \frac{k}{H \cdot W} \right\rfloor, \qquad r = k \bmod (H \cdot W), \qquad h = \left\lfloor \frac{r}{W} \right\rfloor, \qquad w = r \bmod W. \tag{2}$$

1.2 Pseudocode

Algorithm 1: Forward Flattening

Input: Tensor I of shape $B \times C \times H \times W$
Output: Tensor F of shape $B \times (CHW)$

```
Allocate F of size B × (C · H · W)
for b = 0 to B − 1 do
    for c = 0 to C − 1 do
        for h = 0 to H − 1 do
            for w = 0 to W − 1 do
                k ← c · (H · W) + h · W + w
                F[b, k] ← I[b, c, h, w]
return F
```

Algorithm 2: Inverse Reconstruction

Input: Tensor F of shape $B \times (CHW)$, (B, C, H, W)
Output: Reconstructed tensor $\hat{I}$ of shape $B \times C \times H \times W$

```
Allocate Î of size B × C × H × W
for b = 0 to B − 1 do
    for k = 0 to C · H · W − 1 do
        c ← ⌊k/(H · W)⌋
        r ← k mod (H · W)
        h ← ⌊r/W⌋,   w ← r mod W
        Î[b, c, h, w] ← F[b, k]
return Î
```

2 Manual Indexing Example ($B = 1, C = 2, H = 2, W = 3$)

For $B = 1, C = 2, H = 2, W = 3$, the total number of elements is $1 \times 2 \times 2 \times 3 = 12$. Here, the channel stride is $H \cdot W = 6$ and the row stride is $W = 3$, yielding:

$$k = 6c + 3h + w, \qquad c = \lfloor k/6 \rfloor, \quad r = k \bmod 6, \quad h = \lfloor r/3 \rfloor, \quad w = r \bmod 3.$$

For example:

- Element $(b = 0, c = 0, h = 1, w = 2)$: $k = 6(0) + 3(1) + 2 = 5 \implies F[0, 5] = I[0, 0, 1, 2]$. Conversely, from $k = 5$: $c = \lfloor 5/6 \rfloor = 0$, $r = 5 \bmod 6 = 5$, $h = \lfloor 5/3 \rfloor = 1$, $w = 5 \bmod 3 = 2$.

- Element $(b = 0, c = 1, h = 0, w = 1)$: $k = 6(1) + 3(0) + 1 = 7 \implies F[0, 7] = I[0, 1, 0, 1]$. Conversely, from $k = 7$: $c = \lfloor 7/6 \rfloor = 1$, $r = 7 \bmod 6 = 1$, $h = \lfloor 1/3 \rfloor = 0$, $w = 1 \bmod 3 = 1$.
- Element $(b = 0, c = 1, h = 1, w = 2)$: $k = 6(1) + 3(1) + 2 = 11 \implies F[0, 11] = I[0, 1, 1, 2]$. Conversely, from $k = 11$: $c = \lfloor 11/6 \rfloor = 1$, $r = 11 \bmod 6 = 5$, $h = \lfloor 5/3 \rfloor = 1$, $w = 5 \bmod 3 = 2$.

Table 1: Complete address mapping for the test case $B = 1, C = 2, H = 2, W = 3$.

BCHW (b, c, h, w)	**Forward** $k = 6c + 3h + w$	**2D** (b, k)	$I[b, c, h, w]$	**Decoded** (c, h, w)	$\hat{I}[b, c, h, w]$
$(0, 0, 0, 0)$	$6(0) + 3(0) + 0 = 0$	$(0, 0)$	10.0	$(0, 0, 0)$	10.0
$(0, 0, 0, 1)$	$6(0) + 3(0) + 1 = 1$	$(0, 1)$	20.0	$(0, 0, 1)$	20.0
$(0, 0, 0, 2)$	$6(0) + 3(0) + 2 = 2$	$(0, 2)$	30.0	$(0, 0, 2)$	30.0
$(0, 0, 1, 0)$	$6(0) + 3(1) + 0 = 3$	$(0, 3)$	40.0	$(0, 1, 0)$	40.0
$(0, 0, 1, 1)$	$6(0) + 3(1) + 1 = 4$	$(0, 4)$	50.0	$(0, 1, 1)$	50.0
$(0, 0, 1, 2)$	$6(0) + 3(1) + 2 = 5$	$(0, 5)$	60.0	$(0, 1, 2)$	60.0
$(0, 1, 0, 0)$	$6(1) + 3(0) + 0 = 6$	$(0, 6)$	70.0	$(1, 0, 0)$	70.0
$(0, 1, 0, 1)$	$6(1) + 3(0) + 1 = 7$	$(0, 7)$	80.0	$(1, 0, 1)$	80.0
$(0, 1, 0, 2)$	$6(1) + 3(0) + 2 = 8$	$(0, 8)$	90.0	$(1, 0, 2)$	90.0
$(0, 1, 1, 0)$	$6(1) + 3(1) + 0 = 9$	$(0, 9)$	100.0	$(1, 1, 0)$	100.0
$(0, 1, 1, 1)$	$6(1) + 3(1) + 1 = 10$	$(0, 10)$	110.0	$(1, 1, 1)$	110.0
$(0, 1, 1, 2)$	$6(1) + 3(1) + 2 = 11$	$(0, 11)$	120.0	$(1, 1, 2)$	120.0

3 Tensor Dimensions and Measured Reconstruction Errors

The implementation was evaluated on single-channel **MNIST** images ($C = 1, H = 28, W = 28$), three-channel **CIFAR-10** images ($C = 3, H = 32, W = 32$), and synthetic multi-channel intermediate DNN feature maps with $B = 2, H = 16, W = 16$ across $C \in \{8, 16, 32, 64, 128, 256, 500\}$. Reconstruction accuracy is evaluated using the Maximum Absolute Error ($E_{\max}$) and Mean Absolute Error (MAE):

$$E_{\max} = \max_{b,c,h,w} \left| I[b, c, h, w] - \hat{I}[b, c, h, w] \right|, \qquad E_{\text{MAE}} = \frac{1}{BCHW} \sum_{b,c,h,w} \left| I[b, c, h, w] - \hat{I}[b, c, h, w] \right|.$$

Table 2: Experimental results showing $E_{\max}$ and MAE for different classes of image tensors.

Dataset/Input	B	C	H	W	**Flattened Shape** $B \times (CHW)$	$E_{\max}$	**MAE**
MNIST / sample	2	1	28	28	2×784	0.0	0.0
CIFAR / sample	2	3	32	32	2×3072	0.0	0.0
Synthetic (C=8)	2	8	16	16	2×2048	0.0	0.0
Synthetic fmap1	2	16	16	16	2×4096	0.0	0.0
Synthetic (C=32)	2	32	16	16	2×8192	0.0	0.0
Synthetic fmap2	2	64	16	16	2×16384	0.0	0.0
Synthetic fmap3	2	128	16	16	2×32768	0.0	0.0
Synthetic (C=256)	2	256	16	16	2×65536	0.0	0.0
Synthetic fmap4	2	500	16	16	2×128000	0.0	0.0

4 Discussion of Observed Results

1. **Zero Reconstruction Error:** Across all tested configurations ($C \in \{1, 3, 8, 16, 32, 64, 128, 256, 500\}$), both $E_{\max}$ and MAE are identically 0.0. Since $0 \leq w < W$ and $0 \leq h < H$, Euclidean division guarantees a one-to-one bijection between the 3D spatial-channel tuple (c, h, w) and the 1D linear index $k = c(HW) + hW + w$.
2. **Bit-Exact Data Preservation:** Because the flattening and reconstruction operations only re-index memory locations without performing any floating-point arithmetic on the tensor values themselves, no rounding or precision loss occurs, and $\hat{I}$ is bit-for-bit identical to I.